

Lab 11 - Encoded PowerShell Investigation

SOC Analyst Portfolio Lab

Objective

Investigate a harmless PowerShell command executed through the -EncodedCommand parameter, decode the Base64 content, verify PowerShell Script Block Logging telemetry, and assess the event from a SOC analyst perspective.

Environment

Component	Use in Lab
Windows 11 VM	Endpoint used to generate and inspect PowerShell telemetry
PowerShell	Creation, Base64 encoding, execution, and decoding
Windows Event Log	Microsoft-Windows-PowerShell/Operational
Event ID 4104	PowerShell Script Block Logging evidence

Test Procedure

- Confirmed that PowerShell Script Block Logging was enabled (EnableScriptBlockLogging = 1).
- Created the harmless command: Write-Output "LAB11 Encoded PowerShell Test".
- Converted the command to Base64 using PowerShell Unicode encoding.
- Executed the Base64 value through powershell.exe -EncodedCommand.
- Verified that the command executed successfully and printed the expected test text.
- Queried Event ID 4104 and confirmed that Windows recorded the decoded script block.
- Decoded the Base64 value manually and compared it with the Event ID 4104 script content.

Key Evidence

Evidence	Observed Result	Analyst Meaning
Encoded execution	powershell.exe -EncodedCommand <Base64 value>	PowerShell accepted and executed the encoded input.
Decoded content	Write-Output "LAB11 Encoded PowerShell Test"	The Base64 content was harmless and interpretable after decoding.
Event ID 4104	Decoded script block recorded in the PowerShell Operational log	Script Block Logging exposed the underlying command despite encoded execution.
Event ID 600	PowerShell provider lifecycle event; CommandLine field blank	This event did not add useful encoded-command evidence for this test.

Evidence Screenshot - Event ID 4104

The screenshot below shows the encoded command workflow and Event ID 4104 displaying the decoded script block.

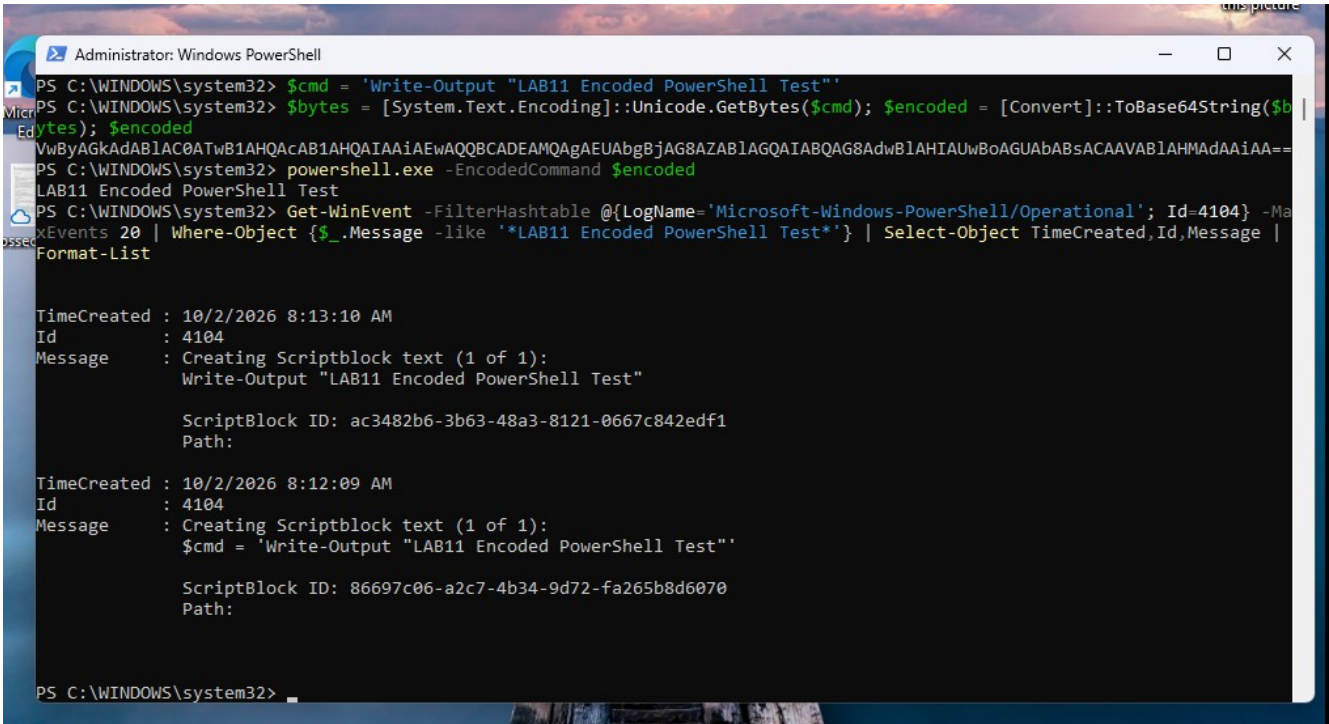


Figure 1. Encoded PowerShell execution and Script Block Logging evidence.

SOC Analyst Triage

Field	Assessment
Severity	Low
Verdict	Benign / authorized test activity
Escalation	No
Primary reason	Decoded content contained only a harmless Write-Output command, with no payload download, credential access, persistence, or destructive behavior.

The use of Base64 and the -EncodedCommand switch can be suspicious because attackers frequently use encoding to reduce readability or evade simple detections. However, encoding by itself is not proof of malicious behavior. The encoded content must be decoded and evaluated in context.

MITRE ATT&CK Mapping

Technique ID	Technique	Application in This Lab
T1059.001	Command and Scripting Interpreter: PowerShell	The test demonstrates PowerShell command execution. The mapping describes behavior, not malicious intent.

Detection and Investigation Lessons

- PowerShell -EncodedCommand should be investigated, but it should not automatically be classified as malicious.
- Base64 is an encoding mechanism, not encryption; analysts can decode it to inspect the underlying command.
- PowerShell commonly expects -EncodedCommand content to be Base64-encoded UTF-16LE / Unicode text.
- Event ID 4104 is valuable because Script Block Logging can reveal the decoded PowerShell content.

- Analysts should correlate decoded content with user context, process activity, network activity, persistence, and other endpoint telemetry before escalating.

Additional Event Review

A Windows PowerShell Event ID 600 was also reviewed. It showed PowerShell provider activity but did not contain the encoded command line in this case, so it was not used as primary evidence.

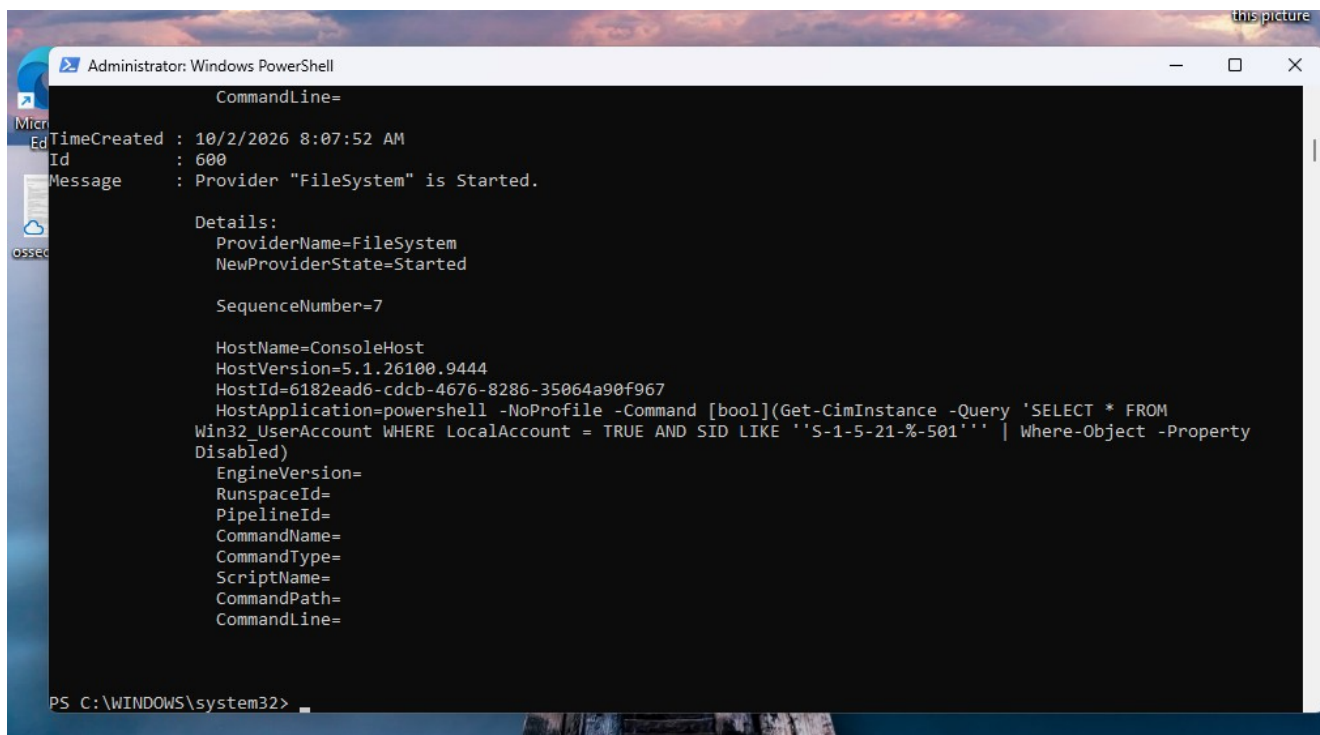


Figure 2. Event ID 600 reviewed as supplementary evidence.

Conclusion

The lab successfully demonstrated how an encoded PowerShell command can be generated, executed, decoded, and investigated. Windows Event ID 4104 captured the underlying script content, allowing the analyst to determine that the activity was benign. The final classification was low-severity authorized test activity with no escalation required.

Safety note: The command used in this lab only printed a test string. No malware, persistence, credential theft, download, or destructive action was performed.